

THE UNPROMPTED

The **AI-Native** App Engineering &
Production Promptbook

Ejajul Seikh



OPEN SOURCE
EDITION



From idea, architecture, database, backend and frontend
to security, scale, release and operations

THE UNPROMPTED

The AI-Native App Engineering & Production Promptbook

Ejajul Seikh

PUBLIC TECHNICAL HANDBOOK

THE UNPROMPTED

The AI-Native App Engineering & Production Promptbook

Ejajul Seikh

From idea, architecture, database, backend and frontend to security, scale, release and operations

Open Source Edition · First Edition · 2026

Copyright, Scope & Open Source Note

Copyright © 2026 Ejajul Seikh. Open Source Edition.

This book is released as an open work under the Creative Commons Attribution 4.0 International license (CC BY 4.0). You may share and adapt the material with appropriate attribution.

This handbook is provided for educational and informational purposes. It does not constitute legal, regulatory, security, privacy, accounting, financial or other professional advice.

The guidance is practical and stack-neutral, but every team should verify important decisions against current primary sources and adapt the material to its actual product, jurisdiction, architecture, providers and risk profile.

This work is shared in good faith, without warranties or guarantees of compliance, fitness for a specific purpose or production suitability. Use professional review where the stakes are material.

Dedication

For the builders who can imagine a product before they know all the vocabulary - and who are willing to replace guesswork with evidence as the product becomes real.

Acknowledgements

This handbook stands on the work of many open engineering and security communities. Particular acknowledgement is due to the maintainers and contributors behind OWASP application and AI security projects, NIST secure software and AI risk-management guidance, W3C accessibility standards, PCI Security Standards Council materials, software supply-chain initiatives such as SLSA, SPDX and CycloneDX, and the broader SRE, open-source and software-testing communities.

It also acknowledges the practical lessons of AI-assisted software development: powerful models can generate useful code at extraordinary speed, but durable software still depends on clear ownership, explicit contracts, secure policy enforcement, data integrity, tests, observability and recoverability.

Foreword - The New Builder

A generation of people can now create software without first spending years learning every framework, command and design pattern. That is a profound expansion of who gets to build. It is also easy to misunderstand what has changed.

AI has lowered the cost of producing code. It has not lowered the cost of being wrong in production.

The difficult parts of serious software were never only syntax. They were deciding what the system means, which state is authoritative, how money and permissions behave under concurrency, what happens when a provider fails, how old clients migrate, what gets logged, who can recover a broken workflow, and whether the team can prove what happened during an incident.

This book is written for the new builder who wants the speed of vibe coding without accepting the fragility of vibe architecture. It starts before the first repository is created, explains the terminology that AI often assumes you already know, shows how models and context actually work, and then builds a disciplined path through product requirements, frontend, backend, databases, security, testing, release and operations.

The central idea is simple: **use AI for acceleration, but use evidence for truth.**

Preface - How to Use This Book

You can read this book in three modes.

Build mode: Start with Part I and move sequentially. At the end of each chapter, create the referenced artifact in your repository. Do not move to the next production stage merely because the UI looks complete.

Audit mode: If you already have an AI-generated app, begin with Part VII and Prompt 13 (whole-repository evidence inventory), then Prompt 14 (canonicalization), Prompt 15 (schema drift), and Prompt 12 for each critical feature. Convert findings into work packages with Prompt 32.

Reference mode: Use the prompt library and glossary as a field manual while working in ChatGPT, Codex, Claude, Gemini, Cursor, Copilot, local models or other AI coding environments. The methodology is deliberately model-agnostic.

A useful rule throughout the book is **inspect -> reason -> change -> verify**. When evidence is missing, record the state as unknown rather than allowing the model to fill the gap with confidence.

The 12-Phase Idea-to-Production Roadmap

01 Idea & System Brief

02 PRD, journeys & state machines

03 Domain & data model

04 Architecture & ADRs

05 Repo, auth, DB & API foundations

06 Vertical-slice feature delivery

07 Security, privacy & compliance mapping

08 Reliability, observability & scale

09 Tests, migrations & CI/CD

10 Staging rehearsal

11 Go / no-go & rollout

12 Operate, recover & improve

The phases are sequential in evidence, not necessarily in calendar time. For example, security design begins during architecture, but the final security audit cannot be completed until the actual implementation exists.

Contents

Part I - From an Idea to an Engineering System	.
1. The Production Maturity Ladder	13
2. Turn the App Idea into a System Brief	14
3. Software Terminology for Vibe Coders	15
4. Choose Architecture by Constraints, Not Fashion	16
Part II - How AI Coding Actually Works	18
5. What a Large Language Model Is Doing	18
6. Tokens, Context Windows and Attention	19
7. Context Engineering: The Real Superpower	20
8. Prompt Engineering vs Context Engineering	21
9. Agents, Tools, Retrieval and Repository Work	22
10. AI Failure Modes You Must Engineer Around	23
Part III - Product, UX and Requirements Before Implementation	24
11. Product Requirements That an AI Can Implement	24
12. User Journeys, State Machines and Edge Cases	25
13. UX, Design Systems and Accessibility	26
14. Data and Domain Discovery Before UI Generation	26
Part IV - Build the Technical Foundation	28
15. Repository Bootstrap and Engineering Guardrails	28
16. Frontend Architecture That Does Not Collapse at Scale	29
17. Backend Domain Architecture	30
18. API Contracts and Compatibility	30
19. Database Engineering: Integrity Before Convenience	31
20. Authentication, Authorization and Tenancy	32
21. Files, Media, Search, Caching and Background Work	33
Part V - Production Hardening	34
22. Configuration, Secrets and Environment Separation	34

23. Security Engineering and Threat Modeling	34
24. Privacy, Data Governance and Compliance Engineering	35
25. Money, Billing and Financial Integrity	36
26. External Integrations and Vendor Boundaries	36
27. Resilience, Backup and Disaster Recovery	37
28. Observability, SLOs and Incident Detection	37
29. Performance, Scalability and Cost Engineering	38
30. Testing as a Risk-Control System	39
31. CI/CD and Software Supply Chain	39
32. Safe Migrations, Deprecation and Old Clients	40
Part VI - Release, Operate and Improve	41
33. Staging as a Production Rehearsal	41
34. Deployment, Rollout and Rollback	41
35. Admin and Support as a Safe Control Plane	42
36. Incident Response, On-Call and Postmortems	42
37. Analytics, Experimentation and Product Telemetry	43
38. Technical Debt, Canonicalization and Engineering Governance	43
Part VII - The Vibe Coding Operating System	45
39. The Evidence-First Development Loop	45
40. The Vertical Slice Rule	45
41. Multi-Agent Orchestration Without Chaos	46
42. Context Handoffs Across Chats, Models and Sessions	47
43. A Production Definition of Done	47
Part VIII - The Reusable Prompt Library	49
How to Use the Prompt Library	49
Part IX - Checklists, Artifacts and Reference Material	71
Appendix A - Required Production Artifacts	71
Appendix B - Launch-Day Checklist	72
Appendix C - The Minimum AI Context Packet	72

Appendix D - Reference Architecture Diagram	73
Appendix E - Glossary	73
Appendix F - Current Reference Baseline	79
Appendix G - Final Principle	80

Prompt Library Index	
01 App idea to System Brief	26 Scalability and capacity audit
02 Product requirements generator	27 Resilience and disaster-recovery audit
03 Architecture options and ADR	28 Test architecture audit
04 Repository bootstrap	29 CI/CD and supply-chain audit
05 Domain model discovery	30 Configuration and feature-flag audit
06 Database schema design	31 Safe migration planner
07 Backend architecture implementation plan	32 Remediation program generator
08 API contract design	33 Controlled implementation
09 Frontend architecture plan	34 Adversarial post-implementation review
10 Authentication design	35 Staging production rehearsal
11 Authorization and tenant-isolation model	36 Final go/no-go gate
12 Feature vertical-slice audit	37 Admin control-plane audit
13 Whole-repository evidence inventory	38 Incident-response readiness
14 Canonicalization and duplication audit	39 Cost and abuse audit
15 Schema drift audit	40 AI feature safety audit
16 Root-cause bug fixer	41 Accessibility and production UX audit
17 Security threat-model audit	42 Documentation and governance audit
18 Secrets and cryptography audit	43 Large ZIP/repository intake prompt
19 Privacy and data-governance audit	44 Context compression and handoff
20 Jurisdiction compliance applicability	45 Code review prompt
21 Payment and money integrity audit	46 Refactor without behavior drift
22 External provider reliability audit	47 Performance investigation
23 Order/workflow distributed state-machine audit	48 Mobile release audit
24 Catalog/inventory/pricing audit	49 Web production audit
25 Observability and SLO audit	50 Master idea-to-production orchestrator

PART I

From an Idea to an Engineering System

Vibe coding becomes dependable engineering when an idea is translated into explicit users, states, data, contracts, risks and operating rules.

1. The Production Maturity Ladder

A prototype proves that an interaction is possible. An MVP proves that a small group of users may value it. A production system must prove something much harder: that the product remains correct, secure, recoverable, observable and operable when real users, real money, real failures and real attackers arrive.

Use this maturity ladder to keep expectations precise:

Level	Name	What it proves	Typical missing pieces
L0	Sketch	The idea can be described or mocked.	Real data, integrations, auth, failure states.
L1	Prototype	A happy path can execute.	Persistence, correctness, security, tests.
L2	MVP	A narrow user group can use core flows.	Hardening, operational recovery, scale evidence.
L3	Launchable	A controlled public launch is defensible.	Higher-scale resilience, mature automation.
L4	Production-grade	Core flows are deterministic, monitored, auditable and recoverable.	Multi-region or advanced governance may remain.
L5	Scale-ready	Capacity, cost, deployment and failure behavior have evidence at expected growth.	Specialized regulatory or global requirements.
L6	High-assurance	Strong governance, separation of duties, compliance evidence and rehearsed recovery support high-impact operation.	Continuous improvement never ends.

The mistake in AI-assisted development is to use **code volume as a proxy for maturity**. A repository may contain hundreds of screens and endpoints while still being L1 if authorization is client-side, data migrations are unsafe, retries duplicate money, operators cannot recover stuck states, or nobody knows which of three competing modules is canonical.

A serious project therefore tracks maturity by evidence. For every material capability, ask:

1. What user or system outcome is required?
2. Which component is the canonical owner?

3. What state changes are allowed?
4. What data is read or written?
5. Which identity and authorization rules apply?
6. What happens during timeout, retry, partial failure and concurrency?
7. How is the outcome tested and observed?
8. How can an operator safely diagnose and recover it?
9. How is the change deployed, migrated and rolled back?

Rule: Production readiness is not a final polishing phase. It is an engineering property accumulated from the first architecture decision.

2. Turn the App Idea into a System Brief

Before selecting a framework, turn the idea into a compact **System Brief**. The brief is the first durable context artifact for both humans and AI agents.

A strong brief contains:

- **Problem:** What costly or frustrating situation exists today?
- **Primary users:** Who experiences it? Separate customers, operators, administrators, partners and machine actors.
- **Core outcomes:** What must users be able to accomplish?
- **Business model:** Subscription, marketplace, transaction fee, internal tool, advertising, licensing, services, or another model.
- **Critical data:** Identity, money, health, location, messages, business records, documents, telemetry and derived data.
- **Risk class:** Low-impact utility, commercial system, financial system, regulated data, safety-related workflow, or privileged control plane.
- **Surfaces:** Web, mobile, desktop, admin, public API, partner API, background workers, hardware or embedded clients.
- **Integrations:** Payments, identity, messaging, maps, storage, analytics, AI, logistics or sector-specific providers.
- **Expected scale:** Active users, tenants, requests per second, jobs, storage growth, files, geographic regions and peak patterns.
- **Availability expectation:** What downtime is tolerable? Which flows must fail closed or degrade gracefully?
- **Launch jurisdictions:** Laws, taxes, consumer rules and data obligations depend on where and how the product operates.
- **Explicit non-goals:** What is intentionally out of scope for the first release?

The System Brief should fit within a few pages. Its purpose is not to solve architecture. Its purpose is to stop the AI from inventing a different product every time the context changes.

The first decomposition

Translate the brief into five maps:

1. **Actor map** - people and systems that can act.
2. **Capability map** - what each actor can do.
3. **Domain map** - identity, catalog, orders, messaging, billing, reporting and other business responsibilities.
4. **Data map** - authoritative records and their owners.
5. **Journey map** - end-to-end flows such as sign-up, purchase, approval, fulfillment or account recovery.

These maps become the seed for the product requirements document, architecture, authorization policy, test plan and observability design.

3. Software Terminology for Vibe Coders

You do not need a computer-science degree to build serious software, but you do need a precise vocabulary. Ambiguous words create ambiguous prompts and ambiguous architecture.

Frontend is the software the user directly interacts with: a web application, mobile app, desktop app or device UI. **Backend** is the server-side logic that validates requests, enforces policy, coordinates business rules, stores data and integrates with external systems. **Database** is persistent structured storage. **Object storage** stores files such as images, videos and documents. **Cache** stores temporary copies to reduce latency or load.

An **API** is a contract through which one component requests behavior or data from another. A **route/endpoint** is a specific API address and method. A **service** is a logical unit that performs a business responsibility. A **repository/data-access layer** isolates persistence operations. A **domain invariant** is a business rule that must always remain true, such as “a refund cannot exceed the captured amount.”

A **transaction** groups data changes so they succeed or fail together. **Idempotency** means safely repeating an operation without duplicating the result. A **race condition** occurs when timing between concurrent operations produces an invalid outcome. **Optimistic concurrency control** detects that data changed after it was read, commonly using a version number or timestamp.

A **queue** stores work for asynchronous processing. A **worker** consumes queued work. A **dead-letter queue** retains jobs that repeatedly fail. An **event** describes something that happened; a **command** requests that something happen. An **outbox pattern** records a state change and the event that must be published in the same transaction, reducing “DB succeeded but message was lost” failures.

Authentication (authn) proves who an actor is. **Authorization (authz)** decides what that actor may do. **RBAC** grants permissions through roles. **ABAC** also uses attributes and context such as tenant, branch, ownership, amount or risk. **Tenant isolation** prevents one customer organization from reading or changing another tenant’s data.

CI automatically builds and tests changes. **CD** automates or governs deployment. **Infrastructure as Code (IaC)** defines infrastructure in version-controlled configuration. **SLO** is a reliability target measured through **SLIs**. **RPO** is the maximum tolerable data-loss window; **RTO** is the target time to restore service.

The glossary at the end of this book expands these terms. When prompting AI, prefer exact terminology over phrases like “make it enterprise” or “make it scalable.” Ask for measurable properties and evidence.

4. Choose Architecture by Constraints, Not Fashion

There is no universally “best” stack. The right architecture is the simplest architecture that satisfies the product’s constraints while leaving a credible path for growth.

A new product usually benefits from a **modular monolith**: one deployable backend with explicit domain modules, a single authoritative data model where appropriate, and clean interfaces between modules. This is easier to test, deploy, observe and change than premature microservices. Microservices become useful when independent scaling, team ownership, fault isolation, data sovereignty or deployment autonomy justify their operational cost.

A stack decision should compare:

- team skill and hiring availability;
- ecosystem maturity and security support;
- mobile/web requirements;
- latency and real-time needs;
- relational versus document data characteristics;
- reporting and analytics complexity;
- deployment target and cost model;
- observability and debugging quality;
- long-term maintenance, not only generation speed;
- interoperability and exit strategy.

For databases, choose by access patterns and integrity needs. Relational systems are strong when relationships, constraints, transactions and reporting matter. Document databases are useful when aggregate-shaped data changes frequently and relationships are looser. Key-value stores excel at fast lookups and ephemeral state. Search engines are secondary indexes for discovery, not usually the source of truth.

For frontends, choose frameworks that support your required platforms, accessibility, testability, secure storage, performance and team workflow. Avoid allowing the AI to introduce a second state-management or routing framework because one screen was easier to implement with it.

Architecture Decision Record (ADR)

For every consequential choice, save a short ADR:

- Context and problem
- Options considered
- Decision
- Reasons and constraints
- Consequences and trade-offs
- Security/data implications
- Revisit trigger

This prevents future AI sessions from “discovering” a different architecture and silently creating V2 beside V1.

Part II - How AI Coding Actually Works

AI can accelerate discovery, design, implementation and verification, but only when its probabilistic nature and context limits are treated as engineering constraints rather than magic.

5. What a Large Language Model Is Doing

A large language model (LLM) generates a sequence of tokens by estimating which token is likely to come next given the context it can currently see. During training, the model learns statistical representations of language, code, patterns, relationships and problem-solving behavior. During use, it does not open a hidden perfect copy of your project; it reasons from the information provided in the current context plus whatever tools and retrieval systems the product exposes.

This has several practical consequences for software work.

First, **fluency is not proof**. The model can produce a confident explanation that is internally coherent but inconsistent with the repository. Second, the model may recognize common patterns and fill missing details with plausible defaults. That is valuable for brainstorming but dangerous for audits, security decisions and data migrations. Third, the model's output quality depends heavily on the quality, relevance and organization of the context.

AI is strongest when it can operate in a closed evidence loop:

inspect -> form a hypothesis -> retrieve exact evidence -> plan -> change -> run tools -> observe results -> revise -> record evidence.

This is very different from “describe the app and ask for all the code.”

Three kinds of knowledge

When an AI coding system responds, separate:

1. **Model knowledge** - patterns learned during training; useful but potentially stale.
2. **Provided context** - text, files, screenshots, schemas, logs and instructions visible now.
3. **Tool evidence** - files actually read, tests actually executed, database schemas actually inspected, current documentation actually retrieved.

For production engineering, tool evidence should dominate any decision that can break security, data, money or compatibility.

Operational principle: Ask AI to prove what exists before asking it to improve what exists.

6. Tokens, Context Windows and Attention

AI systems process text and code as **tokens**, not as human pages. A token may be a whole short word, part of a long word, punctuation, whitespace pattern or code fragment. The **context window** is the finite amount of tokenized material the model can consider in one interaction.

A large context window helps, but it does not eliminate context engineering. A million tokens of unstructured repository content can be less useful than fifty thousand tokens containing the exact contracts, schemas, call graph, failing tests and architecture decisions for the task at hand.

Think of context as an expensive working set with four competing consumers:

- system and project instructions;
- conversation history;
- retrieved code/documents/tool results;
- space reserved for reasoning and the final answer.

When the context is overloaded, several failure modes appear: important details are buried; duplicate modules are confused; outdated findings remain influential; the agent spends attention on irrelevant files; or the response becomes shallow because too little room remains for reasoning and output.

Context budget

For a deep task, intentionally budget context:

- 5-10%: product brief, architecture rules and current task;
- 10-20%: authoritative contracts, schemas and decision records;
- 40-60%: task-specific code and runtime evidence;
- 10-20%: test output, logs and findings;
- reserve the rest for reasoning, implementation description and verification.

The percentages are not a hard law. The point is that **relevance density** matters more than raw capacity.

Attention is not a database query

Even when a model can technically “see” an entire large file, exact recall of every line is not guaranteed. For critical facts, retrieve the exact section again at the moment it is needed. For long projects, store durable facts in repository documents rather than assuming conversation history will remain authoritative.

7. Context Engineering: The Real Superpower

Context engineering is the practice of deciding what the model should see, in what structure, at what stage of the task, and how important facts survive across sessions.

Use a layered context model:

1. **Project Constitution** - non-negotiable engineering rules: canonical ownership, security boundaries, testing requirements, coding conventions and “do not bypass” policies.
2. **System Brief** - users, business model, scope, scale, jurisdictions and critical flows.
3. **Architecture Pack** - system diagrams, domain map, data model, API conventions, authorization model and ADRs.
4. **Task Packet** - exact feature/bug, affected surfaces, expected behavior, evidence, files and acceptance tests.
5. **Live Evidence** - current code, logs, test output and runtime observations.
6. **Handoff Record** - what changed, what passed, what remains, next safe step.

Recommended repository files:

```
/docs/ai/PROJECT_CONSTITUTION.md
/docs/ai/SYSTEM_BRIEF.md
/docs/architecture/SYSTEM_CONTEXT.md
/docs/architecture/DOMAIN_OWNERSHIP.md
/docs/architecture/DATA_MODEL.md
/docs/architecture/API_CONVENTIONS.md
/docs/architecture/AUTHORIZATION_MODEL.md
/docs/decisions/ADR-*.md
/docs/quality/FINDINGS_REGISTER.md
/docs/quality/PRODUCTION_READINESS.md
/docs/operations/RUNBOOKS.md
/docs/ai/HANDOFF.md
```

The files are not bureaucracy. They are **externalized memory**. They let a human or AI session reconstruct the project without relying on a fragile conversation transcript.

Retrieval strategy for a large repo

Do not dump the entire repository first. Start with manifests, directory maps, package files, route registries, schemas, migrations and architecture docs. Then search by domain concepts and symbols. For a feature, trace from entry points outward. For a bug, trace from the observed failure inward until the first incorrect assumption is found.

At every stage, distinguish **authoritative context** from **supporting context**. A database migration is usually more authoritative about schema than an old frontend type. A server-side policy is more authoritative about permission than a hidden button. A live provider webhook payload is more authoritative than a guessed example.

8. Prompt Engineering vs Context Engineering

A prompt is an instruction. Context is the evidence and constraints that make the instruction meaningful. Strong AI engineering requires both.

A good production prompt normally contains:

- **Role:** what disciplines the AI must apply;
- **Objective:** the measurable outcome;
- **Scope:** repositories, apps, domains or files in bounds;
- **Evidence rule:** what must be inspected before conclusions;
- **Constraints:** architecture, security, compatibility and operational rules;
- **Method:** discovery -> analysis -> plan -> implementation -> verification;
- **Required outputs:** findings, decisions, tests, migrations, runbooks or files;
- **Stop conditions:** what the AI must not claim without evidence.

Weak prompt:

Make the backend scalable and production ready.

Stronger prompt:

Audit the order-creation vertical slice from client submission through API, authorization, transaction boundaries, inventory reservation, payment linkage, events, retries and observability. Do not implement until you have identified the canonical owner, current state machine, concurrency risks and schema constraints. For every P0/P1 finding provide exact file/symbol/schema evidence, then create dependency-ordered remediation work packages with tests and rollback.

The stronger prompt does not merely sound technical. It constrains the agent's epistemology: what counts as proof and what sequence is allowed.

Prompt layering

Maintain reusable blocks instead of rewriting everything each time:

- global engineering rules;
- security and data rules;
- project-specific architecture rules;
- task-specific instructions;
- output schema.

This reduces contradictory instructions and makes sessions reproducible.

9. Agents, Tools, Retrieval and Repository Work

An AI **agent** is a model operating in a loop with tools and state. Tools may include file search, code editing, shell commands, tests, browsers, issue trackers, databases, cloud consoles or deployment systems. The important boundary is that the model proposes and selects actions, while deterministic software should enforce authorization, validation and safety.

For repository work, a capable agent should be able to:

- inspect the real file tree;
- search symbols and references;
- read bounded files instead of assuming content;
- edit files in-place;
- run formatters, type checks, builds and tests;
- inspect generated artifacts;
- compare before/after behavior;
- preserve a handoff record.

Do not confuse “connected to a repository” with “understands the repository.” Understanding must be constructed by tracing architecture and evidence.

Safe agent loop

1. Resolve the task and acceptance criteria.
2. Load project constitution and architecture rules.
3. Discover relevant files and current behavior.
4. Identify canonical owner and data/state invariants.
5. Create a change plan with expected files.
6. Implement a bounded work package.
7. Run deterministic verification.
8. Review diffs adversarially.

9. Update evidence and handoff documents.
10. Stop if required evidence cannot be obtained.

For high-risk actions such as production mutations, refunds, permission changes or secret rotation, the authorization layer must live outside the model. Human approval, policy engines, signed workflows and constrained tools should govern impact.

10. AI Failure Modes You Must Engineer Around

AI-assisted coding has recurring failure patterns. Treat them like known reliability hazards.

Hallucinated repository facts. The model names a file, column or endpoint that does not exist. Countermeasure: require exact path/symbol/schema evidence.

Patch-first behavior. The visible error is hidden or bypassed without fixing the authoritative cause. Countermeasure: trace the vertical slice and identify the first wrong assumption.

V2 proliferation. Instead of understanding an existing module, the model creates `serviceV2`, `newApi`, or another state store. Countermeasure: require canonicalization decisions before new implementations.

Security erosion for convenience. A permission, validation or feature gate is removed to make a screen work. Countermeasure: explicitly prohibit weakening controls; repair entitlement or policy models instead.

Schema drift. Generated code references fields that were never migrated, or migrations are written without considering deployed clients. Countermeasure: cross-check code against authoritative migrations and create expand-migrate-verify-contract sequences.

Happy-path completion claims. The feature works once locally but retries, cancellation, offline behavior, reconciliation and operator recovery are absent. Countermeasure: define completion with state machines, failure modes, tests, telemetry and recovery.

Test theater. The model adds tests that mirror its own implementation without testing the risk. Countermeasure: derive tests from invariants and adversarial scenarios.

Context amnesia. Later sessions undo earlier architecture decisions. Countermeasure: maintain ADRs, domain ownership and handoff files in the repo.

False certainty. Static review is described as penetration testing, legal certification or production proof. Countermeasure: label evidence levels and explicitly record unknowns.

Part III - Product, UX and Requirements Before Implementation

Production quality begins with a precise model of users, journeys and state. This part turns product intent into buildable and testable requirements.

11. Product Requirements That an AI Can Implement

A Product Requirements Document (PRD) should describe outcomes and rules without prematurely locking the implementation. For each capability define the actor, trigger, preconditions, main flow, alternate flows, data, permissions, failure states and success measure.

Use this structure for each feature:

```
Capability:  
Actors:  
User problem:  
Trigger / entry point:  
Preconditions:  
Main success flow:  
Alternate flows:  
Invalid operations:  
Data created/read/changed:  
Permissions and scope:  
External dependencies:  
Failure/retry/recovery behavior:  
Notifications:  
Admin/support operations:  
Telemetry:  
Acceptance criteria:  
Out of scope:
```

Acceptance criteria should be observable. “Fast” is vague; “p95 API latency under the agreed load threshold in staging” can be measured. “Secure” is vague; “every resource read is authorized server-side against tenant scope and tested for cross-tenant access” can be verified.

Functional vs non-functional requirements

Functional requirements describe behavior: create order, upload document, approve refund. Non-functional requirements describe system qualities: latency, availability, durability, privacy, accessibility, recovery, auditability, scalability and maintainability.

AI-generated apps often over-invest in functional breadth and under-invest in non-functional depth. Add a non-functional requirements section to every major feature before implementation.

12. User Journeys, State Machines and Edge Cases

Screens are not the product; journeys and state transitions are the product.

For a meaningful workflow, draw a **state machine**. An order might move through `draft` -> `quoted` -> `payment_pending` -> `confirmed` -> `accepted` -> `fulfilled` -> `completed`, with cancellation and refund branches. Each transition needs an actor, preconditions, authority, data changes, side effects and invalid-transition behavior.

For every state machine, answer:

- What is the source of truth?
- Which transitions are user-triggered, system-triggered or provider-triggered?
- Can the same event arrive twice?
- Can events arrive out of order?
- What happens if the client goes offline after submitting?
- What happens if the server commits but the response is lost?
- Which transitions require transactions?
- How are conflicting actions resolved?
- How does support recover a stuck state?
- Which metrics reveal that the machine is unhealthy?

Edge cases should be designed, not discovered by customers. Typical classes include duplicate taps, stale forms, deleted resources, revoked permissions, timezone boundaries, empty data, huge data, partial provider failures, slow networks, retries, concurrent edits, expired sessions and old app versions.

A useful rule

For every happy-path action, brainstorm at least:

1. input failure;
2. authorization failure;
3. network failure;
4. dependency failure;
5. concurrency failure;
6. partial-success failure;
7. duplicate/retry behavior;

8. operator recovery.

This small discipline radically improves AI implementation prompts.

13. UX, Design Systems and Accessibility

Production UX is not merely visual polish. It prevents incorrect operations, reduces support burden and makes the system usable under real conditions.

Build a small design system early: semantic colors, typography, spacing, radii, elevation, icons, motion, layout breakpoints, form controls, buttons, navigation, cards, tables, dialogs, notifications, loading states and empty/error states. Use reusable components, but do not turn every two-line element into an abstraction.

For each interactive surface, design:

- loading, empty, error and partial states;
- form validation with clear recovery;
- disabled and permission-denied states;
- destructive confirmations;
- optimistic updates only where rollback is safe;
- touch, keyboard and screen-reader behavior;
- text scaling and responsive layout;
- network degradation and retry;
- localization-safe text and formats;
- time zones, currencies and addresses.

Accessibility should be part of component acceptance criteria. A design system that gets focus order, labels, contrast, target sizes and error association right can propagate accessible behavior across the entire product.

For high-risk admin or financial actions, UX should deliberately slow the user down: show scope, before/after impact, reason code, approval state and reversibility. “One-click everything” is not a virtue when mistakes are expensive.

14. Data and Domain Discovery Before UI Generation

AI tools often start with screens because screens are visible. A stronger workflow starts with the **domain model**.

List the core nouns in the product: User, Organization, Project, Product, Subscription, Order, Invoice, Message, Device, Document, or whatever is specific to the app. Then distinguish:

- **entity** - has a durable identity and lifecycle;
- **value object** - defined by its attributes, such as Money or Address;
- **aggregate** - a consistency boundary around related entities;
- **event** - something meaningful that happened;
- **policy** - a rule deciding whether an action is allowed;
- **projection/read model** - data shaped for a particular view or query.

For each entity, document owner, identifier, lifecycle, tenant scope, mutable fields, immutable history, retention, deletion, audit needs and key queries.

Only after this should the UI be generated. Otherwise the frontend invents data shapes that later force awkward backend models and migrations.

Part IV - Build the Technical Foundation

This part establishes the repository, backend, APIs, database, identity and asynchronous foundations that every production feature will depend on.

15. Repository Bootstrap and Engineering Guardrails

A professional repository should communicate how the system is built before an AI edits it.

Create, at minimum:

- deterministic package lockfiles;
- formatting and lint rules;
- static type checking where the ecosystem supports it;
- test commands separated by layer;
- environment templates without real secrets;
- a clear source directory structure;
- a migration mechanism;
- CI configuration;
- ownership documentation;
- a security policy and dependency update process;
- architecture and AI instruction documents.

A typical multi-surface repository may look like:

```
/apps/  
  web/  
  mobile/  
  admin/  
/services/  
  api/  
/packages/  
  contracts/  
  ui/  
  domain-shared/  
/infra/  
/migrations/  
/docs/  
  architecture/  
  decisions/  
  operations/  
  ai/
```

The exact structure matters less than explicit boundaries. Avoid “shared” folders that become dumping grounds. Shared packages should own stable, genuinely cross-cutting contracts or primitives.

Set pre-commit or CI gates so generated code cannot silently bypass basic quality: formatting, lint, typecheck, tests, secret scanning and migration validation.

16. Frontend Architecture That Does Not Collapse at Scale

Frontend architecture should separate presentation from server communication, local state and business rules.

A robust feature usually contains:

- route/screen entry points;
- UI components;
- view-model/controller/state layer;
- API client or repository abstraction;
- typed request/response models;
- validation and error mapping;
- tests.

Keep server-authoritative rules on the server. The client may pre-validate for usability, but the backend must still enforce permission, money, inventory, ownership and state-transition rules.

Design state by ownership:

- **server state** - fetched records, lists and mutations; use query caching with explicit invalidation;
- **local interaction state** - open tabs, field focus, temporary input;
- **persistent local state** - carefully chosen preferences or offline queues;
- **derived state** - recompute from canonical inputs rather than storing another copy.

Common AI-generated problems include multiple networking clients, duplicated DTOs, feature-specific auth logic, unbounded global stores, manual cache updates and screen files containing business logic. Audit these early.

Mobile clients need additional attention to app lifecycle, secure storage, deep links, background execution limits, permissions, offline behavior, slow networks, app-version compatibility and release signing.

17. Backend Domain Architecture

A backend should make business rules obvious and difficult to bypass.

A practical request path is:

```
route/controller
-> authentication
-> authorization/policy
-> request validation
-> application use case/service
-> domain rules
-> repository/transaction
-> events/jobs/integrations
-> response mapper
```

Controllers should translate transport concerns, not contain the core business model. Data-access code should not decide whether an actor is allowed to perform an operation. Domain services should not silently reach into unrelated modules.

For every mutation define:

- command/input contract;
- actor and tenant scope;
- preconditions;
- idempotency semantics;
- transaction boundary;
- concurrency strategy;
- state transition;
- events/side effects;
- audit event;
- response and error codes.

Use dependency injection or explicit module composition to make external providers replaceable and testable. Avoid global mutable singletons where lifecycle and configuration become invisible.

A modular monolith can enforce strong boundaries with modules, interfaces and database ownership without the network complexity of microservices. If services are later split, clear domain seams make the migration safer.

18. API Contracts and Compatibility

An API is a public promise between components. Treat it like a versioned product.

A route contract should define method, path, authentication, authorization scope, request schema, response schema, error envelope, pagination, idempotency, rate limits and retry safety.

Prefer machine-readable schemas where practical. Generate clients or validate both sides against the same contract, but keep business logic independent from transport-generated code.

For errors, return stable machine-readable codes in addition to human messages. A client should be able to distinguish `PERMISSION_DENIED`, `STALE_VERSION`, `VALIDATION_ERROR`, `PAYMENT_REQUIRED` and `DEPENDENCY_UNAVAILABLE` without parsing English strings.

Compatibility rule

Assume old clients will remain deployed. For breaking changes, use:

expand -> migrate/backfill -> make consumers compatible -> verify -> contract/remove.

Do not rename a database field and update one app in the same release if older apps still depend on it. Use deprecation telemetry to prove old behavior is no longer used before deletion.

19. Database Engineering: Integrity Before Convenience

A database is not only storage. It is one of the last defenses against corrupted state.

For each business-critical entity define:

- authoritative primary identifier;
- tenant/organization ownership;
- required and nullable fields;
- uniqueness rules;
- foreign keys or reference integrity;
- indexes for real query patterns;
- timestamps and time-zone conventions;
- money representation;
- audit/history requirements;
- soft-delete/archive semantics;
- retention and deletion behavior;
- transaction and concurrency requirements.

Use constraints for invariants the database can enforce. Application validation alone is vulnerable to race conditions and alternate code paths.

Migrations

Every schema change should be repeatable, reviewed and deployable against real existing data. A safe migration includes:

1. preflight query and backup/restore confidence;
2. expand schema in a backward-compatible way;
3. deploy code that can handle both states if needed;
4. backfill with resumable, idempotent jobs;
5. verify row counts, constraints and business invariants;
6. switch reads/writes;
7. observe;
8. remove obsolete fields only after compatibility evidence.

Never treat “migration ran successfully” as proof that data is correct. Validate domain invariants after the migration.

20. Authentication, Authorization and Tenancy

Authentication answers **who are you?** Authorization answers **may you perform this action on this resource in this scope right now?**

Build identity as a lifecycle: registration, verification, login, session issuance, refresh/rotation, logout, password or credential recovery, device/session management, MFA or step-up, account lock, offboarding and compromise response.

Authorization should be server-side and resource-aware. A strong policy evaluates:

```
actor + action + resource + tenant scope + role/permissions + contextual conditions
```

Do not trust a `tenant_id`, `organization_id`, `branch_id` or owner ID simply because the client sent it. Derive or validate scope against the authenticated actor and the resource being accessed.

RBAC is convenient for common job functions. ABAC adds conditions such as resource ownership, branch, amount, time, environment or risk. Most serious systems use a hybrid.

For privileged actions, add step-up authentication, reason codes, four-eyes approval where warranted, immutable audit evidence and safe reversal paths.

Test authorization adversarially: user A reading user B; tenant A enumerating tenant B; branch manager accessing another branch; revoked staff using cached permissions; support impersonation; invitation privilege escalation; bulk exports; object-level IDOR.

21. Files, Media, Search, Caching and Background Work

Non-database infrastructure creates many production surprises.

Files/media: upload through constrained flows, validate type and size, scan or safely process untrusted content, store outside the application filesystem, use signed URLs where appropriate, separate public from private objects, and define deletion/retention. Never trust a client-provided MIME type or filename as security evidence.

Search: treat the search index as a projection of canonical data. Define indexing events, reindex/backfill procedures, deletion propagation and behavior when the index lags or is unavailable.

Caching: every cache needs an ownership and invalidation strategy. Avoid caching authorization decisions longer than their security semantics allow. Measure hit rate and stale-data impact.

Background jobs: jobs need identifiers, retry policy, backoff/jitter, idempotency, timeouts, concurrency limits, dead-letter handling and observability. Store enough state to answer “was this job completed, retried, abandoned or manually recovered?”

Use an outbox or equivalent reliable handoff when a database mutation must cause an event. Avoid untracked fire-and-forget work after returning an HTTP response.

Part V - Production Hardening

Once the core architecture exists, production engineering makes it secure, resilient, observable, testable, scalable and economically controllable.

22. Configuration, Secrets and Environment Separation

Configuration should be explicit, validated and environment-aware. Secrets should never be treated like ordinary configuration.

Separate local, test, staging and production. Each environment should have independent credentials and clearly different provider endpoints. Production startup should fail closed when required security configuration is missing.

Secret management requirements include:

- no committed credentials;
- no secrets embedded in web/mobile bundles;
- least-privilege service identities;
- rotation and revocation procedures;
- versioned keys where cryptography requires migration;
- restricted CI access;
- redaction in logs and error reports;
- documented ownership and expiry.

Feature flags need lifecycle governance: owner, reason, default, environments, exposure rules, kill-switch behavior and removal date. Flags that live forever become hidden architecture branches.

23. Security Engineering and Threat Modeling

Security is the systematic removal of unjustified trust.

Start with a threat model: assets, actors, entry points, trust boundaries, privileged operations, external providers and abuse cases. Then evaluate credible attack paths such as broken access control, injection, XSS, SSRF, unsafe uploads, command execution, insecure deserialization, credential theft, token replay, webhook forgery, mass assignment, sensitive-data exposure and resource exhaustion.

For every high-risk attack path write an attack narrative:

Attacker capability -> entry point -> missing control -> state/data reached -> impact

Then identify the exact control that should break the chain and the test that proves it.

Security controls should include secure defaults, input validation, output encoding, authorization, rate limits, secrets management, encryption where appropriate, dependency governance, secure headers, CSRF/CORS controls where relevant, audit logging and incident response.

Static analysis and dependency scanning are useful signals, not penetration-test proof. Production security requires layered verification and continuous monitoring.

24. Privacy, Data Governance and Compliance Engineering

Privacy engineering begins with a data inventory, not with a privacy-policy page.

For each data category record:

- source and collection point;
- purpose;
- storage location;
- owner and processors/subprocessors;
- sensitivity classification;
- access roles;
- sharing/export paths;
- retention decision;
- deletion/anonymization workflow;
- backup implications;
- user rights/request workflow where applicable;
- encryption and logging exposure.

Do not collect data “just in case.” Minimize duplicated PII and avoid copying sensitive values into logs, analytics or support tools.

Compliance must be mapped to the actual business model and jurisdiction. Build a **compliance applicability matrix** rather than pasting a generic checklist:

```
Requirement -> applicability -> product control -> backend/data control  
-> operational process -> evidence -> owner -> open legal question
```

AI should retrieve current primary sources for jurisdiction-specific claims. It should not self-certify legal compliance. Use qualified legal, privacy and security review where the interpretation matters.

25. Money, Billing and Financial Integrity

Any system that moves money needs stronger invariants than ordinary CRUD.

Represent money with explicit currency and safe precision, often integer minor units or a fixed decimal type. Define tax, fee, discount and rounding order once. Snapshot historical commercial terms on the transaction so later price changes do not rewrite history.

Trace the complete money lifecycle:

quote -> order -> payment intent -> authorization/capture -> provider result -> webhook -> ledger -> refund/chargeback -> settlement/payout -> invoice/report -> reconciliation.

Critical controls include:

- idempotent payment initiation;
- verified webhook signatures and replay protection;
- a payment state machine separate from but reconciled with order state;
- partial refund rules;
- immutable ledger/audit entries;
- negative-balance and hold rules;
- provider reconciliation;
- administrative adjustments with reasons and approval;
- deterministic recovery when provider state and internal state disagree.

Treat any path that can create, lose or duplicate money without deterministic reconciliation as a launch blocker.

26. External Integrations and Vendor Boundaries

Every external provider is a distributed-system boundary. Assume it can be slow, unavailable, inconsistent, rate-limited or changed without your deployment.

Hide provider-specific logic behind a canonical adapter. For each provider define:

- credentials and environment endpoints;
- timeout;
- retry/backoff and retry safety;
- idempotency keys;
- rate limits and quotas;
- circuit breaker or degraded mode where useful;

- webhook verification and replay defense;
- schema/version compatibility;
- reconciliation/backfill jobs;
- dead-letter/manual recovery;
- data shared and privacy implications;
- monitoring and alerting;
- provider replacement strategy for critical dependencies.

Do not scatter direct SDK calls across business logic. A coherent boundary makes failure behavior testable and vendor switching possible.

27. Resilience, Backup and Disaster Recovery

Resilience is the ability to continue or recover when components fail. Design for failure explicitly.

Perform failure-mode analysis for database, cache, queue, object storage, DNS/network, secrets manager, external providers and each critical service. Ask what fails, how quickly it is detected, whether retries are safe, what data can be inconsistent, and how the operator restores correctness.

Backups are not real until restores have been tested. Define:

- what is backed up;
- frequency and retention;
- encryption and access;
- geographic/provider separation;
- RPO and RTO targets;
- restore procedure;
- integrity verification after restore;
- recovery after accidental deletion, bad deploy, migration, ransomware or credential compromise.

A disaster-recovery runbook should be executable by someone who did not write the system.

28. Observability, SLOs and Incident Detection

Observability answers whether the system is working, why it is not, who is affected and whether recovery succeeded.

Use three technical signals together: logs, metrics and traces. Add business signals such as successful signups, payments, order completion, message delivery or whatever defines product health.

Every request should carry a correlation/request identifier through services, queues and provider calls where practical. Logs should be structured and redact secrets/PII. Metrics should expose latency, throughput, errors, saturation and domain-specific failures. Traces show where distributed time was spent.

Define SLIs and SLOs for critical journeys. For example, “successful checkout creation” is more meaningful than raw CPU usage. Alerts should be actionable: route to an owner, include severity and context, and avoid paging for symptoms nobody will act on.

Build dashboards from user journeys outward, not infrastructure inward.

29. Performance, Scalability and Cost Engineering

Scalability is not “use Kubernetes.” It is the ability to meet workload and latency expectations as demand changes without losing correctness or creating uncontrolled cost.

Estimate major workloads: reads, writes, search, media, queues, reports, scheduled jobs and external calls. Identify hot tenants, hot keys, expensive queries and fan-out paths.

Audit:

- horizontal-scaling assumptions;
- database connection pools and indexes;
- pagination and unbounded scans;
- N+1 queries;
- caches and invalidation;
- queue throughput and backpressure;
- worker concurrency;
- media transformation and CDN delivery;
- provider quotas;
- real-time connection scaling;
- large exports/reports;
- cost amplification from retries or abuse.

Do not invent performance numbers. Define load tests with representative data and measure. Track capacity thresholds and unit economics so growth does not become an operational surprise.

30. Testing as a Risk-Control System

Test count and coverage percentage are weak proxies for production confidence. Build tests around invariants and failure modes.

A mature test portfolio includes:

- unit tests for pure domain rules;
- database and repository integration tests;
- API contract tests;
- authorization and tenant-isolation tests;
- component/widget tests;
- end-to-end tests for critical journeys;
- concurrency/idempotency tests;
- migration tests against realistic old data;
- provider/webhook tests;
- failure injection and recovery tests;
- security regression tests;
- load/performance tests;
- accessibility tests;
- backup/restore rehearsal.

For every P0/P1 defect, add a regression test that would have caught it. When AI writes a fix and its tests, ask a second review to invent adversarial cases the implementation author may have missed.

31. CI/CD and Software Supply Chain

A production release is an artifact produced through a chain of trust from source code to deployment.

CI/CD should verify formatting, linting, type checking, unit/integration tests, security scans, migrations and build reproducibility. Lock dependencies. Restrict CI tokens. Pin or govern third-party actions. Produce immutable artifacts and keep provenance that links them to source and build inputs.

Maintain an SBOM using a standard format such as SPDX or CycloneDX where appropriate. Scan dependencies and container images for known vulnerabilities, but combine scanner output with reachability and risk assessment.

A strong release pipeline separates environments, requires appropriate approvals, protects production credentials, performs controlled migration ordering and supports canary, rolling, blue-green or equivalent rollout with explicit rollback triggers.

Supply-chain security is especially important in AI-assisted coding because agents can introduce new dependencies very quickly. Require justification for every new package and prefer well-maintained, necessary dependencies over convenience libraries for trivial functions.

32. Safe Migrations, Deprecation and Old Clients

Production systems evolve while users are still connected. Compatibility is therefore a feature.

For every breaking candidate change inventory producers, consumers, background jobs, events, database fields and deployed client versions. Then plan a compatibility window.

Use an **expand-migrate-verify-contract** sequence:

1. expand the schema or API without breaking old behavior;
2. deploy compatible producers/consumers;
3. backfill and reconcile data;
4. switch traffic or reads/writes gradually;
5. verify telemetry and invariants;
6. contract/remove old paths after evidence shows they are unused.

Feature flags should have owners and expiry dates. Deprecation should have telemetry. Delete only after the old path is proven inactive.

Part VI - Release, Operate and Improve

The job is not finished at merge. This part turns code into a controlled service with staging proof, rollback, support recovery and continuous learning.

33. Staging as a Production Rehearsal

Staging should be similar enough to production to expose deployment, migration, configuration and integration defects. It does not need identical scale, but it must exercise the same architecture and security assumptions.

A production rehearsal should include:

- clean deployment from the release artifact;
- migration from a realistic previous database snapshot;
- representative roles and tenant data;
- critical journeys end-to-end;
- provider success, failure, delay and duplicate callbacks;
- network and dependency failure;
- queue backlog and recovery;
- restart during in-flight work;
- expired/rotated credentials;
- backup and restore;
- alert delivery and trace correlation;
- rollback of application and compatible data changes;
- old-client compatibility where relevant.

Record objective pass/fail evidence. An untested high-risk recovery path remains unknown, not “probably fine.”

34. Deployment, Rollout and Rollback

A release plan should answer exactly what will change, how users are exposed, what operators will watch and when the change will be reversed.

Before release, confirm artifact identity, configuration, migrations, feature flags, dashboards, alerts, support messaging, owners and rollback commands. Use progressive delivery for risky changes: internal users, small percentage, selected tenants, region, then broader exposure.

Define rollback triggers in advance. Examples include authorization anomalies, data-integrity errors, payment reconciliation drift, elevated crash rate or critical latency regression.

Rollback is not always “deploy old code.” A database migration may need forward-fix semantics. This is why expand-contract changes are safer than irreversible destructive migrations.

35. Admin and Support as a Safe Control Plane

A mature system lets operators diagnose and recover production without direct database editing.

For each realistic failure - stuck order, missing payout, locked account, failed webhook, bad product state, delivery reassignment, corrupted cache - provide read-only diagnostics and constrained recovery actions.

Dangerous admin actions need:

- least-privilege permission;
- tenant/resource scope;
- step-up authentication when appropriate;
- confirmation and impact preview;
- reason code;
- approval for high-impact changes;
- idempotency;
- before/after audit record;
- reversible design or explicit irreversibility;
- rate/batch limits;
- reconciliation verification.

The admin panel is not a CRUD generator. It is a privileged production control plane.

36. Incident Response, On-Call and Postmortems

Incidents will happen. Operational maturity is measured by how quickly the team detects, contains, communicates and learns from them.

Create playbooks for credential compromise, account takeover, cross-tenant exposure, data corruption, payment inconsistency, provider outage, malicious webhook, mass export, bad deployment and lost or leaked secrets.

Each playbook should cover:

detection -> triage -> containment -> evidence preservation -> rotation/revocation -> recovery -> impact assessment -> legal/regulatory escalation -> communication -> postmortem.

Postmortems should be blameless but precise. Document timeline, contributing conditions, failed controls, detection gaps, recovery effectiveness and concrete follow-up work with owners and deadlines.

On-call ownership must be explicit. Alerts without an owner are noise.

37. Analytics, Experimentation and Product Telemetry

Analytics should help answer product questions without becoming an uncontrolled copy of user data.

Define an event taxonomy with stable names, properties, ownership and privacy classification. Avoid sending raw sensitive payloads into analytics by default. Version event semantics when meanings change.

Separate operational telemetry from product analytics. Operational metrics answer “is the system healthy?” Product analytics answer “are users achieving the intended outcome?”

For experiments, define hypothesis, population, assignment, guardrail metrics, primary metric, duration and stopping rules. Do not let experimentation override safety, authorization or legal obligations.

Instrument user journeys from the PRD so telemetry can validate whether the production system behaves as designed.

38. Technical Debt, Canonicalization and Engineering Governance

Every successful product accumulates debt. AI-assisted products can accumulate it faster because creating a parallel implementation is cheap.

Maintain a **canonical ownership map** for domains, APIs, data models and shared components. When two implementations overlap, write a canonicalization decision: current options, evidence, chosen owner, migration path, deprecation path, tests and delete-after conditions.

Use a persistent findings register with severity, status, evidence, owner, remediation, verification and residual risk. This turns repeated AI audits into a cumulative program instead of rediscovering the same issues.

Governance should be lightweight but enforceable: ADRs for major choices, schema-change procedure, definition of done, security requirements, feature-flag lifecycle, dependency policy, release checklist and AI-agent rules.

Part VII - The Vibe Coding Operating System

This part turns AI-assisted development into a repeatable engineering workflow that can survive large repositories, long projects and multiple agents.

39. The Evidence-First Development Loop

Use one loop for features, defects, refactors and audits:

1. **Discover** - inventory what exists.
2. **Reconstruct** - describe the current end-to-end behavior from evidence.
3. **Classify** - verified, partial, duplicated, inconsistent, dead/unwired, stub/mock, missing or unknown.
4. **Define target** - canonical owner, state machine, contracts and controls.
5. **Plan** - dependency-ordered work packages.
6. **Implement** - one bounded package at a time.
7. **Verify** - tests, builds, migrations and runtime checks.
8. **Adversarial review** - try to break the result.
9. **Record** - update findings, ADRs and handoff.
10. **Operate** - monitor after release.

For severity, use a release-oriented model:

- **P0:** stop launch - data loss, privilege bypass, unreconciled money, severe security exposure, irreversible migration risk or broken core flow.
- **P1:** must fix before production - material correctness, privacy, reliability, recovery or operational control gap.
- **P2:** bounded risk that can be fixed after initial launch with an explicit workaround and owner.
- **P3:** maintainability, optimization or polish.

The labels matter because they force prioritization. AI tends to produce long flat lists; operations need ordered risk.

40. The Vertical Slice Rule

A feature is not implemented because a screen exists, an endpoint exists or a table exists. A production feature is a complete vertical slice.

Trace:

```

UI / entry point
-> client state and validation
-> API contract
-> authentication
-> authorization / tenant scope
-> domain invariants / state machine
-> database transaction and constraints
-> events / jobs
-> external integrations
-> notification / audit
-> observability
-> tests
-> failure / retry / recovery
-> privacy / compliance implications

```

For every feature, identify the source of truth at each layer and the contracts between layers. If two apps implement different business rules, treat it as an inconsistency until one canonical rule is chosen.

A vertical-slice audit is the best antidote to vibe-coded “almost complete” systems because it exposes disconnected implementations and invisible failure states.

41. Multi-Agent Orchestration Without Chaos

Multiple AI agents can increase throughput, but parallelism is safe only when work boundaries are explicit.

Use an orchestrator or human lead to maintain the dependency graph. Parallelize independent work such as documentation, isolated tests, separate UI components or non-overlapping provider adapters. Serialize changes that touch canonical data models, migrations, shared contracts, auth policies or the same core domain.

Each agent should receive:

- the Project Constitution;
- the relevant ADRs and domain ownership;
- a bounded work package;
- files it may change;
- invariants it must preserve;
- required tests;
- a handoff template.

Require agents to re-read the current files before editing because another work package may have changed the repository since the original plan.

Do not merge two implementations of the same capability and “clean it later.” Canonicalization must happen before parallel implementation.

42. Context Handoffs Across Chats, Models and Sessions

Long projects should be restartable from the repository itself.

At the end of every substantial session, update a handoff:

```
Task / work package:
Goal:
Canonical owner:
Files changed:
Migrations:
Contracts changed:
Tests run and results:
Runtime verification:
Open findings:
Residual risks:
Rollback:
Next safe action:
Do-not-repeat decisions:
```

When a new session begins, load the handoff, project constitution, relevant ADRs and current files. Do not paste the entire old conversation unless it contains unique evidence not recorded elsewhere.

For large repositories, keep a searchable findings register and architecture index. This reduces context cost and prevents old reasoning from competing with current source code.

43. A Production Definition of Done

A feature should be marked production-complete only when all applicable conditions are true:

- one canonical business/domain implementation is identified;
- all production clients use the intended API/contract;
- server-side authorization and tenant isolation are enforced;
- inputs are validated and sensitive outputs are minimized;
- schema integrity, indexes and migrations are correct;
- state transitions are explicit and invalid transitions rejected;
- retries/duplicates are safe where necessary;
- money and inventory are reconciliable;
- providers have timeout, retry, webhook-security and recovery behavior;
- failure and partial-success states are designed;
- privileged actions are auditable;
- data purpose, access, retention and deletion are defined;
- security threat scenarios are addressed;

- applicable compliance requirements are mapped;
- logs, metrics and traces support diagnosis without leaking sensitive data;
- unit/integration/contract/E2E tests cover critical risks;
- performance has evidence for expected load;
- backup and recovery implications are known and rehearsed where necessary;
- deployment and rollback are documented;
- duplicate/deprecated paths have removal evidence;
- support has controlled recovery actions;
- no P0 blocker remains;
- accepted risk has an owner and review date.

This checklist is intentionally stricter than an MVP definition of done. It is the bridge from “works” to “can be trusted.”

Part VIII - The Reusable Prompt Library

These prompts are designed to be copied into an AI coding environment. Replace bracketed variables and attach only the evidence relevant to the task. For high-risk analysis, require exact paths, symbols, schemas, commands and test output.

How to Use the Prompt Library

Every prompt assumes that the AI can inspect the relevant code or evidence. If the tool cannot access the repository, do not let it pretend otherwise. Attach or retrieve the required materials first.

For deep audits, prepend this shared instruction block:

You are working on a production-bound software system. Act as a principal software architect, senior backend/frontend engineer, database engineer, application-security engineer, SRE, QA lead and data-governance reviewer.

STRICT RULES

1. Treat repository files, schemas, migrations, configuration, tests and runtime behavior as evidence. Do not guess material facts.
2. Trace features end-to-end; UI existence is not proof of implementation.
3. Identify the canonical owner before creating another module, endpoint, table, type, state store or service.
4. Never weaken authentication, authorization, tenant isolation, validation, auditability, financial controls or data protection just to make a flow work.
5. Separate facts from inferences and mark insufficient evidence UNKNOWN.
6. For P0/P1 findings provide exact evidence, failure/attack scenario, affected surfaces, root cause, target state, migration/compatibility, tests, telemetry and rollback/recovery.
7. Do not claim completion if required verification was not executed.

Recommended finding statuses: VERIFIED_IMPLEMENTED , PARTIAL , DUPLICATED , INCONSISTENT , DEAD_OR_UNWIRED , STUB_OR MOCK , MISSING , UNKNOWN .

Prompt 01 - App idea to System Brief

Run when: You have an idea but no durable product/engineering context yet.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: App idea to System Brief

Turn the app idea below into a concise System Brief. Identify actors, problems, core outcomes, business model, surfaces, critical data, external providers, expected scale, risk class, launch jurisdictions, non-goals and the top 10 unknowns that must be resolved before architecture. Do not choose a stack until constraints are explicit.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns

- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 02 - Product requirements generator

Run when: You need buildable requirements before coding.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Product requirements generator

Create a production-minded PRD for [FEATURE/PRODUCT]. For each capability define actors, trigger, preconditions, main flow, alternate flows, invalid operations, data, permissions, integrations, failure/retry/recovery, telemetry and measurable acceptance criteria. Separate functional and non-functional requirements.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 03 - Architecture options and ADR

Run when: You need to choose a stack or system shape.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Architecture options and ADR

Given the System Brief and constraints, compare 3 viable architectures. Evaluate complexity, delivery speed, security, data integrity, scalability, operability, cost, ecosystem maturity and migration path. Recommend the simplest architecture that satisfies the constraints. Produce an ADR with trade-offs and revisit triggers.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 04 - Repository bootstrap

Run when: You are creating the project from zero.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Repository bootstrap

Design the initial repository layout, package boundaries, environment strategy, lint/type/test commands, CI gates, migration system, secrets approach, docs structure and AI context files. Prefer minimal dependencies and explicit ownership. Output the exact initial file/folder plan and bootstrap checklist.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 05 - Domain model discovery

Run when: The product concept exists but the data/business model is vague.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Domain model discovery

Identify the core domains, entities, value objects, aggregates, state machines, invariants, domain events and ownership boundaries. Separate write models from read projections where useful. Mark uncertain concepts and questions rather than inventing business rules.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 06 - Database schema design

Run when: You need the first production-capable schema.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Database schema design

Design the database model from the verified domain model and access patterns. Specify identifiers, tenant ownership, nullability, uniqueness, references, indexes, transactions, concurrency, money

types, timestamps, audit/history, retention and deletion. Include migration strategy and sample integrity checks.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 07 - Backend architecture implementation plan

Run when: You are about to build server-side foundations.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Backend architecture implementation plan

Define route/controller, authn, authz, validation, application-service, domain, repository, event/job and provider boundaries. For every core mutation specify idempotency, transaction and concurrency strategy. Produce a module map and implementation order without creating microservices unless constraints justify them.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 08 - API contract design

Run when: Frontend and backend need a stable interface.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: API contract design

Design canonical API contracts for [CAPABILITY]. Include route, version, authentication, authorization scope, request/response schemas, error codes, pagination, idempotency, retry safety, rate limits, and compatibility rules. Include contract-test cases and deprecation strategy.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry

- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 09 - Frontend architecture plan

Run when: You are building web/mobile UI foundations.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Frontend architecture plan

Design a feature-oriented frontend architecture separating presentation, server state, local interaction state, typed API clients, validation, error mapping and tests. Define routing, auth lifecycle, cache invalidation, offline/poor-network behavior, accessibility and release-mode safety.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 10 - Authentication design

Run when: Identity flows are being introduced or replaced.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Authentication design

Design registration, login, verification, session/token lifecycle, refresh/rotation, logout, recovery, device/session management, MFA/step-up, brute-force controls, secure storage, offboarding and compromised-account response. Identify threat scenarios and required tests.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 11 - Authorization and tenant-isolation model

Run when: The app has organizations, roles, staff or sensitive resources.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Authorization and tenant-isolation model

Reconstruct or design the authorization model using actor + action + resource + tenant scope + context. Define RBAC/ABAC catalog, server enforcement points, privileged actions, invitations, ownership transfer, support impersonation, exports and audit events. Never rely on UI hiding as security.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 12 - Feature vertical-slice audit

Run when: A feature exists but may be partial or inconsistent.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Feature vertical-slice audit

Trace [FEATURE] end-to-end from UI through client state, API, authn/authz, domain rules, DB, jobs/events, providers, notifications, observability, tests and failure/recovery. Reconstruct the current state machine, classify gaps, define the canonical target and only then produce remediation packages.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 13 - Whole-repository evidence inventory

Run when: You do not yet trust your understanding of an AI-generated repo.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Whole-repository evidence inventory

Inventory applications, services, routes, auth, policies, domains, schemas/migrations, jobs, caches, storage, integrations, flags, shared packages, tests, observability, CI/CD, mocks and top duplicate responsibilities. Prove material claims with exact paths/symbols. Do not implement.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry

- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 14 - Canonicalization and duplication audit

Run when: V1/V2 modules or duplicate patterns may exist.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Canonicalization and duplication audit

For each major domain identify competing modules/routes/types/tables/components. Determine the safest canonical owner from evidence. Produce CURRENT OPTIONS -> CHOSEN OWNER -> MIGRATION -> DEPRECATION -> TESTS -> DELETE-AFTER CONDITIONS. Do not add another implementation.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 15 - Schema drift audit

Run when: Code and database may disagree.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Schema drift audit

Cross-check every business-critical field referenced in code against authoritative schema and migrations. Find missing/renamed fields, duplicate identifiers, nullability/default conflicts, missing constraints/indexes and migration gaps. Produce a safe expand/backfill/verify/contract plan.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 16 - Root-cause bug fixer

Run when: You have a visible error and want a correct fix.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Root-cause bug fixer

Do not patch the symptom. Reproduce or trace the failing path to the first incorrect assumption in contract, schema, config, state, policy or code. Search for duplicate legacy implementations. Explain root cause with evidence, define the canonical fix, migrations/compatibility, regression tests, telemetry, then implement only the smallest root-cause work package.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 17 - Security threat-model audit

Run when: You need systematic application security review.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Security threat-model audit

Threat-model assets, actors, trust boundaries and entry points. Test credible paths for broken access control, injection, XSS, SSRF, unsafe uploads, deserialization, command execution, secret leakage, token replay, webhook forgery, mass assignment, supply-chain risk and resource exhaustion. For each P0/P1 path give attack narrative, exact control and verification.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 18 - Secrets and cryptography audit

Run when: Keys, tokens, env files or signing/encryption need review.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Secrets and cryptography audit

Search code, config, history-visible artifacts, CI, mobile/web bundles and scripts for credentials. Evaluate secret manager usage, least privilege, rotation, key versioning, hashing, token signing, randomness, TLS and encryption responsibilities. Produce immediate rotation actions for any plausible exposure.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable

- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 19 - Privacy and data-governance audit

Run when: The app processes user or business data.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Privacy and data-governance audit

Create a data inventory by collection point, purpose, storage, owner/processors, classification, access, sharing, retention, deletion, backup, rights workflow, encryption and logging exposure. Flag unnecessary collection, duplicated PII and deletion gaps. Do not invent legal retention periods.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 20 - Jurisdiction compliance applicability

Run when: You are preparing launch in a specific country/sector.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Jurisdiction compliance applicability

Using current primary authoritative sources, map [JURISDICTION] obligations that may apply to [BUSINESS_MODEL]. For each requirement return APPLIES / MAY APPLY / DOES NOT APPEAR TO APPLY / NEEDS COUNSEL, product/data/operational controls, evidence and exact open legal questions. Do not self-certify compliance.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 21 - Payment and money integrity audit

Run when: The product charges, refunds, settles or pays out money.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Payment and money integrity audit

Trace quote -> order -> payment -> provider -> webhook -> ledger -> refund -> settlement/payout -> invoice/report. Verify currency/precision, tax/discount order, idempotency, payment vs order state, partial refunds, late/duplicate webhooks, reconciliation, admin adjustments and audit trails. Treat unreconciled money as P0/P1.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 22 - External provider reliability audit

Run when: A third-party API is production-critical.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: External provider reliability audit

For each provider inspect canonical adapter, credentials by environment, timeouts, retries/backoff, idempotency, quotas, circuit/degraded mode, webhook signatures/replay, schema drift, outage behavior, reconciliation, dead-letter/manual recovery, data sharing and observability.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 23 - Order/workflow distributed state-machine audit

Run when: Multiple apps/services can change the same workflow.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Order/workflow distributed state-machine audit

Reconstruct all states and transitions for [WORKFLOW] across clients, backend, DB, jobs and providers. Test duplicate submission, lost response, out-of-order events, concurrent actions, partial completion, offline client and admin override. Define one source of truth and reconciliation rules.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns

- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 24 - Catalog/inventory/pricing audit

Run when: The product has SKUs, stock, pricing or branch/location availability.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Catalog/inventory/pricing audit

Audit entity identity, variants/options, inventory source of truth, reservations, oversell races, branch pricing, discount/tax semantics, stale carts, imports, publication/moderation and adjustments. Define exact server-authoritative rules and atomic transitions.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 25 - Observability and SLO audit

Run when: Operators cannot confidently diagnose production.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Observability and SLO audit

For each critical journey determine whether operators can answer is it working, slow, who is affected, what changed, where it failed and whether retry is safe. Audit structured logs, correlation IDs, traces, metrics, business KPIs, queue/DB/provider health, crash reporting, synthetic checks, dashboards and actionable alerts. Define SLI/SLO candidates.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 26 - Scalability and capacity audit

Run when: The system works locally but expected growth is unknown.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Scalability and capacity audit

Given [EXPECTED_SCALE], inspect query patterns, connection pools, hot tenants/keys, caches, queues, worker concurrency, media, search, provider quotas, scheduled jobs and cost amplification. Define representative load tests and capacity thresholds. Do not invent benchmark numbers.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 27 - Resilience and disaster-recovery audit

Run when: You need evidence the service can recover.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Resilience and disaster-recovery audit

Perform failure-mode analysis for DB, cache, queue, object storage, provider, network, secrets and deployment. Audit timeout/retry storms, degradation, idempotency, backpressure, backups, restore evidence, RPO/RT0, accidental deletion, credential compromise and rollback. Produce a DR runbook.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 28 - Test architecture audit

Run when: Tests exist but production confidence is unclear.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Test architecture audit

Map tests to business and security risks: domain invariants, DB integration, authz, API contracts, provider callbacks, UI, E2E, concurrency, migration, failure injection, security regression, load, accessibility and restore. For every P0/P1 gap require a regression test before closure.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state

- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 29 - CI/CD and supply-chain audit

Run when: You need a trustworthy commit-to-production path.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: CI/CD and supply-chain audit

Audit protected branches/reviews, deterministic builds, lockfiles, dependency policy, vulnerability and secret scanning, SAST/DAST where appropriate, SBOM, provenance, container pinning, CI permissions, third-party actions, environment separation, deployment approvals, migration order, artifact immutability and rollback.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 30 - Configuration and feature-flag audit

Run when: Local/staging/prod behavior differs or gates are confusing.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Configuration and feature-flag audit

Find hard-coded URLs/credentials, sandbox/production mix-ups, debug flags, permissive security defaults, unvalidated env vars and feature flags without owners/expiry. Design a typed environment contract and startup validation that fails closed on dangerous missing config.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 31 - Safe migration planner

Run when: A schema/API/domain canonicalization must ship without breaking users.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Safe migration planner

Inventory all producers/consumers and old clients. Create an expand -> migrate/backfill -> verify -> contract plan with resumable scripts, integrity queries, rollback/forward-fix, queue/event compatibility, feature-flag lifecycle and deprecation telemetry.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 32 - Remediation program generator

Run when: Audits are complete and you need an executable plan.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Remediation program generator

Convert verified findings into dependency-ordered work packages: stop-launch risks, canonical foundations, feature completion, reliability/observability, compliance evidence and cleanup. Include exact surfaces, prerequisites, migrations, tests, telemetry, rollout, rollback and completion evidence. Identify changes that must not run in parallel.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 33 - Controlled implementation

Run when: A specific work package is approved.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Controlled implementation

Implement only [WORK_PACKAGE]. Re-verify current files first, state canonical owner/invariants, list expected changes, then edit. Do not weaken security or create duplicates. Add tests and telemetry. Run lint/typecheck/build/tests/migration validation and report exact results, residual risk and rollback.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns

- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 34 - Adversarial post-implementation review

Run when: A fix is claimed complete.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Adversarial post-implementation review

Assume the implementation is wrong. Re-derive behavior and try to prove old paths remain, authz is bypassable, tenants can cross, concurrency corrupts state, retries duplicate effects, migration loses data, old clients break, provider callbacks are replayable, errors are hidden or tests are misleading. PASS only with evidence.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 35 - Staging production rehearsal

Run when: Code review is complete and launch is near.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Staging production rehearsal

Create and execute a staging rehearsal covering clean deploy, realistic migration, rollback, role matrix, critical journeys, provider failure/delay/duplicates, queue backlog, restart during work, rotated credentials, backup/restore, alert delivery, trace correlation and old-client compatibility. Record objective pass/fail.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 36 - Final go/no-go gate

Run when: You believe the product is launch-ready.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Final go/no-go gate

Score core flow correctness, canonical architecture, DB/migrations, authn/authz, security, privacy/compliance, money, integrations, scale, observability, DR, testing, CI/CD, admin control and incident/support readiness. Any open P0 or hard-rule P1/unknown is NO-GO. Output blockers, accepted risks, rollback triggers and 24h/72h/7d monitoring.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 37 - Admin control-plane audit

Run when: Operators need safe production controls.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Admin control-plane audit

Audit admin capabilities for least privilege, tenant scope, step-up auth, reason/approval, dry-run, idempotency, before/after audit, reversibility, export controls, impersonation, batch safety and reconciliation. Identify missing recovery operations that currently require DB edits or scripts.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 38 - Incident-response readiness

Run when: You need operational playbooks before launch.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Incident-response readiness

Create playbooks for credential compromise, account takeover, cross-tenant exposure, payment inconsistency, malicious webhook, database corruption, provider outage, leaked secret and suspicious

export. Cover detection, containment, evidence, rotation, recovery, impact, legal escalation, communications and postmortem.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 39 - Cost and abuse audit

Run when: Usage-based providers or expensive workloads can be abused.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Cost and abuse audit

Find cost-amplification paths in OTP/email, maps, media, AI, search, exports, reports, retries, queues, public queries, storage/bandwidth, coupons/referrals. Define quotas, rate limits, per-tenant budgets, anomaly alerts and kill switches that preserve state correctness.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 40 - AI feature safety audit

Run when: The product itself uses AI/agents.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: AI feature safety audit

Audit prompt injection, indirect injection, tool authorization independent of the model, tenant isolation in retrieval/context, secrets in prompts/logs, autonomous mutations, approval boundaries, deterministic policy, provider retention, hallucinated IDs, idempotency, kill switch, evaluations and audit trail. The model must not be the authorization layer.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry

- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 41 - Accessibility and production UX audit

Run when: Core flows work technically but usability is not production-grade.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Accessibility and production UX audit

Audit semantics/screen readers, keyboard/focus, contrast, text scaling, touch targets, errors, loading/empty/offline/permission states, destructive actions, localization, formats, time zones, addresses and low bandwidth. Separate visual polish from defects that can cause unsafe or incorrect operations.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 42 - Documentation and governance audit

Run when: Future AI sessions keep recreating architecture debt.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Documentation and governance audit

Require system context, domain ownership, API/data contracts, ADRs, migration/deprecation policy, coding boundaries, security requirements, definition of done, release checklist, runbooks, on-call, dependency ownership, feature-flag lifecycle and AI-agent rules. Design lightweight governance that prevents accidental V2/V3 duplication.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 43 - Large ZIP/repository intake prompt

Run when: You are handing a large codebase to an AI session.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Large ZIP/repository intake prompt

First map repository size, apps, package manifests, entry points, route registries, schemas/migrations, shared packages and docs. Create a bounded retrieval plan by domain. Do not read random files or claim whole-repo understanding. Produce an architecture index and the next evidence queries before implementation.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 44 - Context compression and handoff

Run when: The conversation is getting long or a new session/model will continue.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Context compression and handoff

Create a durable handoff containing objective, verified architecture facts, canonical decisions, exact files changed, tests/results, open findings, unknowns, residual risks, rollback and next safe action. Separate facts from inferences. Prefer references to repository docs over copying large raw code.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 45 - Code review prompt

Run when: You need a rigorous review of a change set.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Code review prompt

Review the diff against the stated requirements and architecture. Check correctness, state transitions, authz, data integrity, concurrency, idempotency, compatibility, error handling, observability, test quality, secrets/PII, dependencies and unnecessary complexity. Rank findings by release severity and cite exact changed symbols.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state

- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 46 - Refactor without behavior drift

Run when: A module is hard to maintain but must remain compatible.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Refactor without behavior drift

Before editing, derive current public behavior, callers, contracts, data invariants and tests. Define the canonical target boundary and an incremental refactor plan. Preserve externally observable behavior unless a documented defect is intentionally changed. Add characterization tests before structural changes.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 47 - Performance investigation

Run when: A screen/API/job is slow.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Performance investigation

Measure before optimizing. Trace latency across client, network, backend, DB, cache, queue and provider. Find query counts, payload size, serialization, render/rebuild cost and contention. Produce evidence, bottleneck hypothesis, targeted benchmark, smallest fix, regression threshold and observability.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 48 - Mobile release audit

Run when: You are preparing a mobile app for store release.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Mobile release audit

Audit secure storage, permissions, deep links, lifecycle/background behavior, offline/network loss, push notifications, API compatibility, crash reporting, accessibility, performance, bundle secrets, signing/release config, privacy disclosures and old-version behavior. Trace critical journeys into the backend.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 49 - Web production audit

Run when: You are preparing a web app for public production.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Web production audit

Audit routing/deep links, auth/cookies, CSRF/CORS/origin, XSS/content injection, headers/CSP where applicable, state/query caching, accessibility, responsive behavior, bundle secrets, source maps, error reporting, performance, CDN/caching and API compatibility.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Prompt 50 - Master idea-to-production orchestrator

Run when: You want the full program from a new idea to production.

[PASTE SHARED INSTRUCTION BLOCK WHEN THE TASK IS HIGH-RISK]

CONTEXT: [APP / REPO / FEATURE / ENVIRONMENT]

GOAL: Master idea-to-production orchestrator

Run a phased program rather than one mega-generation: System Brief -> PRD/journeys -> domain/data model -> architecture/ADR -> repo bootstrap -> auth/data/API foundations -> vertical-slice feature implementation -> security/privacy -> resilience/observability -> tests -> CI/CD -> staging rehearsal -> go/no-go -> launch monitoring. At each phase enforce explicit exit criteria and preserve evidence in repository documents.

REQUIRED OUTPUT

- Executive summary
- Evidence / assumptions / unknowns
- Current state
- Canonical target state
- Findings ordered P0 -> P3 when applicable
- Dependency-ordered next actions
- Required tests and telemetry
- Migration / compatibility / rollback notes
- Exact completion evidence

Part IX - Checklists, Artifacts and Reference Material

This final part is designed to be printed, copied into project documents or converted into issue templates.

Appendix A - Required Production Artifacts

A serious product should converge on these artifacts. Not every small app needs a separate document for each item, but the information should exist and be maintainable.

- System Brief and Product Requirements Document
- System context and container/component diagrams
- Domain ownership map
- Canonical data model and data dictionary
- API inventory/contracts
- Authentication/session design
- Permission catalog and authorization matrix
- Critical business state machines
- Data inventory/classification/retention matrix
- Threat model and security control matrix
- External provider inventory and data-sharing map
- Dependency/SBOM inventory
- Compliance applicability matrix and counsel question pack
- P0/P1 risk register
- Migration/deprecation register
- Test matrix by business risk
- SLI/SLO catalog
- Dashboards and alert catalog
- Backup/restore/DR evidence
- Incident-response playbooks
- Admin/support recovery catalog
- Release/rollback runbook
- Launch go/no-go evidence pack
- Post-launch 24h/72h/7d monitoring plan
- Architecture Decision Records
- AI project constitution and current handoff

Appendix B - Launch-Day Checklist

Product and data

- Core journeys have objective staging evidence.
- No open P0; hard-rule P1/unknown risks are resolved or launch is blocked.
- Production configuration and providers are correct.
- Database migration is reviewed, timed and reversible/forward-fixable.
- Backups are current and restore procedure is known.

Security and access

- Production secrets are not shared with local/test environments.
- Admin and privileged access uses least privilege and step-up where appropriate.
- Authorization/tenant-isolation regression tests pass.
- Critical webhooks verify signatures and replay/idempotency.
- Security monitoring and incident contacts are active.

Operations

- Dashboards, SLOs and alerts are live.
- On-call owner and escalation path are clear.
- Support has known-issue and recovery instructions.
- Rollback triggers and commands are written.
- Release owner, incident commander backup and communication channel are defined.

After release

- Compare error rate, latency and business-success metrics to pre-release baseline.
- Watch migration/backfill/reconciliation jobs.
- Sample audit logs and provider callbacks.
- Review 24-hour findings before expanding rollout.

Appendix C - The Minimum AI Context Packet

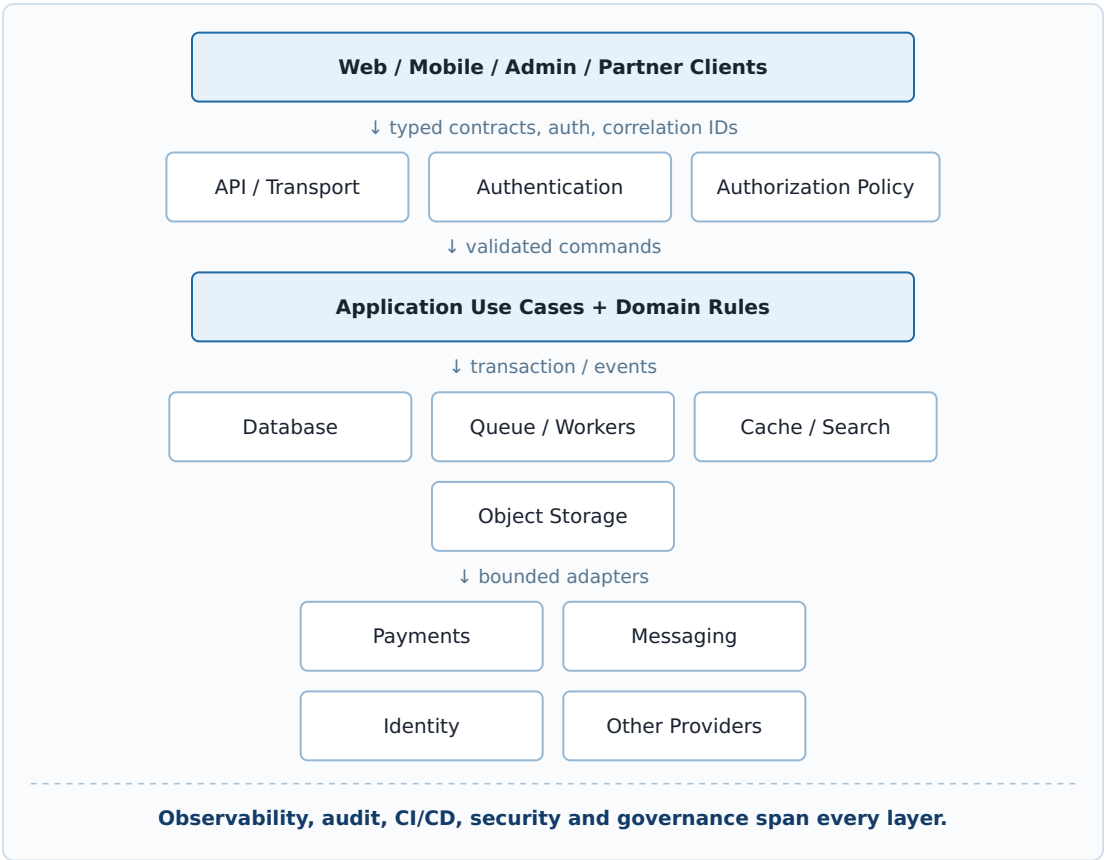
When beginning a new coding session, provide this context in order:

1. **Task statement** - one objective and acceptance criteria.
2. **Project Constitution** - non-negotiable architecture/security rules.
3. **System Brief** - users, model, scale, jurisdictions.
4. **Relevant ADRs/domain ownership** - what is canonical.
5. **Current handoff/findings** - what already happened.

- 6. **Exact files/evidence** - retrieved for this task.
- 7. **Verification commands** - how success will be proven.

Avoid flooding the model with unrelated files. Add context on demand as the evidence path expands.

Appendix D - Reference Architecture Diagram



Appendix E - Glossary

ABAC. Attribute-Based Access Control; authorization that considers properties of the actor, resource, action and context.

ADR. Architecture Decision Record; a short durable record of a significant technical decision and its trade-offs.

API. Application Programming Interface; a defined contract used by software components to communicate.

API gateway. A boundary that can route, authenticate, limit, observe or transform API traffic. It should not become a dumping ground for domain logic.

Artifact. A produced build output such as a container image, mobile package, binary or static web bundle.

Authentication. The process of establishing the identity of a user, service or device.

Authorization. The policy decision that determines whether an authenticated actor may perform an action on a resource.

Backfill. A controlled process that populates or repairs existing records after a schema or business-model change.

Backpressure. A mechanism for slowing producers when consumers or downstream systems cannot safely keep up.

Blue-green deployment. A rollout strategy that keeps old and new production environments available so traffic can be switched between them.

Cache. Temporary storage used to reduce latency or load; it requires explicit freshness and invalidation rules.

Canary release. A progressive rollout to a small percentage of traffic or users before broader deployment.

CD. Continuous Delivery/Deployment; the practices and automation used to promote verified software toward production.

CI. Continuous Integration; automated building and verification of code changes.

Circuit breaker. A resilience pattern that temporarily blocks calls to a failing dependency to prevent cascading failures.

Client. Software that calls another service, often a web app, mobile app, desktop app or backend consumer.

Concurrency. Multiple operations making progress during overlapping time periods.

Constraint. A rule enforced by the database, such as uniqueness, foreign-key or check constraints.

Context window. The finite tokenized information an AI model can consider during an interaction.

Correlation ID. An identifier propagated through a request or workflow so logs and traces can be connected.

CQRS. Command Query Responsibility Segregation; separating write commands from read models when the complexity justifies it.

CRUD. Create, Read, Update, Delete; basic data operations. Business systems generally need stronger domain rules than CRUD alone.

Dead-letter queue. A destination for messages/jobs that cannot be processed successfully after the allowed retry policy.

Deployment. The act of making a built software artifact available in an environment.

Domain. A coherent area of business responsibility such as identity, billing, catalog or fulfillment.

Domain invariant. A rule that must always remain true within a business domain.

DTO. Data Transfer Object; a data shape used across a boundary such as an API.

E2E test. End-to-end test; verifies a complete user or system journey across multiple components.

Encryption at rest. Protection applied to stored data or storage media. Its exact responsibility depends on the platform and threat model.

Encryption in transit. Protection of data while moving across networks, commonly through TLS.

Entity. A domain object with a stable identity and lifecycle.

Error budget. The amount of unreliability permitted by an SLO during a measurement window.

Event. A durable or transient fact indicating that something happened in the system.

Feature flag. A runtime or build-time switch controlling exposure or behavior. Production flags need owners, defaults and expiry.

Foreign key. A relational database constraint linking one record to another authoritative record.

Graceful degradation. Reduced but controlled functionality when a dependency or subsystem fails.

Idempotency. The property that repeating an operation does not duplicate its intended effect.

Index. A database/search structure that accelerates specific query patterns at storage and write cost.

Infrastructure as Code. Version-controlled definitions for infrastructure resources and configuration.

Invariant. A condition that must remain true before and after allowed state transitions.

JWT. JSON Web Token; a compact signed token format. It is a format, not automatically a secure session architecture.

Latency. Time taken to complete an operation, often measured with percentiles such as p50, p95 and p99.

Least privilege. Granting only the permissions necessary for an actor or service to perform its responsibilities.

Ledger. A structured record of financial movements or balances designed for traceability and reconciliation.

LLM. Large Language Model; a model that generates and processes token sequences based on learned statistical representations.

Load test. A controlled test that measures system behavior under representative or stressful workload.

Microservice. An independently deployable service around a bounded responsibility. It adds network and operational complexity.

Migration. A controlled change to data schema, data contents, APIs or architecture across deployed versions.

Modular monolith. A single deployable application containing explicit internal domain/module boundaries.

Multi-tenancy. A system in which multiple customer organizations share infrastructure while their data and permissions remain isolated.

Observability. The capability to infer system state and failure causes from telemetry such as logs, metrics and traces.

Optimistic concurrency. A strategy that detects conflicting updates using a version or timestamp rather than locking for the entire operation.

Outbox pattern. A reliability pattern that records domain state changes and pending events together before asynchronous publication.

P0/P1/P2/P3. A release-oriented severity scheme ranging from stop-launch to lower-risk improvement.

PII. Personally Identifiable Information; terminology and legal scope vary by jurisdiction, so classify using applicable rules.

Policy engine. A component that evaluates authorization or business policy using explicit inputs and rules.

PRD. Product Requirements Document; defines user problems, capabilities, requirements, constraints and acceptance criteria.

Primary key. A database identifier that uniquely identifies a record within its table or collection model.

Rate limit. A rule limiting how frequently an actor, tenant, token or source may perform an operation.

RBAC. Role-Based Access Control; authorization through roles that aggregate permissions.

Reconciliation. The process of comparing two sources of state and resolving discrepancies deterministically.

Regression test. A test created to ensure a previously fixed defect cannot silently return.

Repository. A version-controlled collection of source code, configuration, tests and documentation.

Retry. A repeated attempt after failure. Retries must have limits, backoff and idempotency semantics.

Rollback. Returning to a previous safe application or configuration state. Data migrations may require forward-fix instead.

RPO. Recovery Point Objective; maximum acceptable amount of recent data loss after a disaster.

RTO. Recovery Time Objective; target time to restore an acceptable service level after an incident.

SBOM. Software Bill of Materials; a machine-readable inventory of software components and related metadata.

Schema. The structural definition and constraints of stored data or a contract.

Schema drift. Mismatch between code assumptions and the real data schema/migrations deployed.

SDK. Software Development Kit; a packaged client or set of tools for integrating with a platform.

Secret. A credential or cryptographic material that grants access or protects sensitive operations.

Service Level Indicator. A measured signal representing reliability or quality, such as successful request ratio or latency.

Service Level Objective. A target for an SLI over a defined period.

Session. Server/client state representing an authenticated interaction over time.

SLI. Service Level Indicator.

SLO. Service Level Objective.

Soft delete. Marking a record as deleted/archived without immediately removing it from storage. It requires query and retention discipline.

Source of truth. The authoritative record or component whose state resolves disagreements elsewhere.

SSRF. Server-Side Request Forgery; an attack in which an attacker induces a server to make unintended network requests.

Staging. A pre-production environment used to rehearse deployment and behavior under production-like configuration.

State machine. An explicit model of allowed states and transitions for a workflow or entity.

Telemetry. Machine-generated signals about software operation, such as logs, metrics, traces and events.

Tenant isolation. Controls that prevent one tenant/customer organization from accessing another tenant's data or actions.

Threat model. A structured analysis of assets, actors, trust boundaries, attacks and controls.

Token (AI). A unit of text/code representation processed by a language model.

Token (auth). A credential or signed artifact representing identity or authorization context; unrelated to AI tokens.

Trace. A representation of an operation as it flows across components, often containing spans and timing.

Transaction. A group of database operations with atomicity and consistency guarantees defined by the database.

Value object. A domain object defined by its values rather than a durable identity, such as Money or DateRange.

Vertical slice. The complete implementation of a capability across UI, API, policy, domain, data, side effects, tests and operations.

Webhook. An inbound HTTP callback from another system. Production handling requires authentication/signature validation and replay/idempotency controls.

Worker. A process that consumes asynchronous jobs or events.

XSS. Cross-Site Scripting; execution or injection of untrusted script/content in a web context.

Zero trust. A security approach that avoids granting implicit trust based solely on network location or prior access.

Appendix F - Current Reference Baseline

The following are authoritative starting points for further reading. Always verify the current version and applicability at the time of use.

- OWASP Application Security Verification Standard (ASVS): <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP Mobile Application Security Verification Standard (MASVS): <https://mas.owasp.org/MASVS/>
- OWASP Large Language Model Security Verification Standard (LLMSVS): <https://owasp.org/www-project-llm-verification-standard/>
- NIST Secure Software Development Framework (SP 800-218): <https://csrc.nist.gov/projects/ssdf>
- NIST AI Risk Management Framework: <https://www.nist.gov/itl/ai-risk-management-framework>
- NIST Generative AI Profile: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- W3C Web Content Accessibility Guidelines (WCAG) 2.2: <https://www.w3.org/TR/WCAG22/>
- PCI Security Standards Council - PCI DSS: <https://www.pcisecuritystandards.org/standards/pci-dss/>
- SLSA software supply-chain specification: <https://slsa.dev/spec/>
- SPDX specifications: <https://spdx.dev/use/specifications/>
- CycloneDX specification: <https://cyclonedx.org/specification/overview/>

Version note for this edition

At the time this edition was prepared, current official materials included OWASP ASVS 5.0, OWASP Top 10 (2025 release), WCAG 2.2, PCI DSS 4.0.1, NIST SSDF SP 800-218 v1.1 as the final SSDF publication while a later revision remained in development, and active software supply-chain specifications in the SLSA/SPDX/CycloneDX ecosystems. Because these evolve, the prompt library intentionally instructs AI to retrieve current primary sources rather than rely on frozen version numbers.

Appendix G - Final Principle

The target is not more code. The target is a system in which every important responsibility has a clear owner, every state transition is controlled, every material data flow is understood, every privileged action is authorized and auditable, failures are recoverable, deployments are reversible, and operators can prove what happened without editing production data by hand.

AI changes the speed of software creation. Engineering discipline determines whether that software deserves to be trusted.